

16. Write a C program that can perform a letter frequency attack on any monoalphabetic substitution cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

```

Q16.cpp
1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  typedef struct {
6      char ch;
7      int count;
8  } Freq;
9
10 void sortFreq(Freq f[], int n) {
11     int i, j;
12     Freq temp;
13
14     for (i = 0; i < n - 1; i++) {
15         for (j = i + 1; j < n; j++) {
16             if (f[j].count > f[i].count) {
17                 temp = f[i];
18                 f[i] = f[j];
19                 f[j] = temp;
20             }
21         }
22     }
23 }
24
25 int main() {
26     char cipher[1000];
27     int top, i, j;
28     int count[26] = {0};
29
30     char english[] = "ETAOINSHRDLUMWFGYPBVKJXQZ";
31
32     printf("Enter ciphertext:\n");
33     fgets(cipher, sizeof(cipher), stdin);
34
35     printf("Enter number of possible plaintexts: ");
36     scanf("%d", &top);
37
38     for (i = 0; cipher[i] != '\0'; i++) {
39         if (isalpha((unsigned char)cipher[i])) {
40             count[toupper(cipher[i]) - 'A']++;
41         }
42     }
43
44     Freq f[26];
45
46     for (i = 0; i < 26; i++) {
47         f[i].ch = 'A' + i;
48         f[i].count = count[i];
49     }
50
51     sortFreq(f, 26);
52
53     printf("\nFrequency order in ciphertext:\n");
54     for (i = 0; i < 26; i++) {
55         printf("%c ", f[i].ch);
56     }
57
58     printf("\n\nPossible plaintexts:\n");
59
60     for (j = 0; j < top; j++) {
61         printf("\nCandidate %d:\n", j + 1);
62
63         for (i = 0; cipher[i] != '\0'; i++) {
64             if (isalpha((unsigned char)cipher[i])) {
65                 char c = toupper(cipher[i]);
66                 int index = c - 'A';
67
68                 if (index < 26)
69                     putchar(english[(index + j) % 26]);
70             } else {
71                 putchar(cipher[i]);
72             }
73         }
74     }
75
76     return 0;
77 }

```

```
Enter ciphertext:
WKH TXLFN EURZQ IRA
Enter number of possible plaintexts: 5

Frequency order in ciphertext:
R E F H I K L N Q A T U W X Z P B J S C G V M D Y O

Possible plaintexts:

Candidate 1:
JLH BXCNM IVYZG RYE

Candidate 2:
XCR VQUSW NKPEY DPT

Candidate 3:
QUD KZMHF SJBTP LBA

Candidate 4:
ZML JEWRG HXVAB CVO

Candidate 5:
EWC XTFDY RQKOV UKI

-----
Process exited after 13.41 seconds with return value 0
Press any key to continue . . . |
```

17. Write a C program for DES algorithm for decryption, the 16 keys (K1, K2, ..., K16) are used in reverse order. Design a key-generation scheme with the appropriate shift schedule for the decryption process.

```

36
37 int PC2[48] = {
38     14,17,11,24,1,5,3,28,15,6,21,10,
39     23,19,12,4,26,8,16,7,27,20,13,2,
40     41,52,31,37,47,55,30,40,51,45,33,48,
41     44,49,39,56,34,53,46,42,50,36,29,32
42 };
43
44 int SHIFTS[16] = {
45     1,1,2,2,2,2,2,2,
46     1,2,2,2,2,2,2,1
47 };
48
49 uint32_t rot28(uint32_t x, int n) {
50     return ((x << n) | (x >> (28 - n))) & 0xFFFFFFFF;
51 }
52
53 void generateKeys(uint64_t key, uint64_t subkey[16]) {
54     uint64_t p = permute(key, PC1, 56, 64);
55
56     uint32_t C = (p >> 28) & 0xFFFFFFFF;
57     uint32_t D = p & 0xFFFFFFFF;
58
59     int i;
60
61     for (i = 0; i < 16; i++) {
62         C = rot28(C, SHIFTS[i]);
63         D = rot28(D, SHIFTS[i]);
64
65         uint64_t cd = ((uint64_t)C << 28) | D;
66         subkey[i] = permute(cd, PC2, 48, 56);
67     }
68 }
69
70 uint32_t feistel(uint32_t R, uint64_t K) {
71     uint64_t x = ((uint64_t)R << 16) | R;
72     return (uint32_t)(x ^ K);
73 }
74
75 uint64_t DES_decrypt(uint64_t block, uint64_t key) {
76     uint64_t subkey[16];
77     uint64_t p;
78     uint32_t L, R, temp;
79     int i;
80
81     generateKeys(key, subkey);
82
83     p = permute(block, IP, 64, 64);
84
85     L = p >> 32;
86     R = p & 0xFFFFFFFF;
87
88     for (i = 15; i >= 0; i--) {
89         temp = R;
90         R = L ^ feistel(R, subkey[i]);
91         L = temp;
92     }
93
94     p = ((uint64_t)R << 32) | L;
95
96     return permute(p, FP, 64, 64);
97 }
98
99 int main() {
100     uint64_t cipher, key, plain;
101
102     printf("Enter ciphertext in hexadecimal: ");
103     scanf("%llx", &cipher);
104
105     printf("Enter 64-bit key in hexadecimal: ");
106     scanf("%llx", &key);
107
108     plain = DES_decrypt(cipher, key);
109
110     printf("Decrypted plaintext = %016llx\n", plain);
111
112     return 0;
113 }

```

```
Enter ciphertext in hexadecimal: 85E813540F0AB405
Enter 64-bit key in hexadecimal: 133457799BBCDFF1
Decrypted plaintext = B103480EC03AD68F
```

```
-----
Process exited after 18.99 seconds with return value 0
Press any key to continue . . . |
```

18. Write a C program for DES the first 24 bits of each subkey come from the same subset of 28 bits of the initial key and that the second 24 bits of each subkey come from a disjoint subset of 28 bits of the initial key.

```

1  #include <stdio.h>
2  #include <stdint.h>
3
4  int PC1[56] = {
5      57,49,41,33,25,17,9,1,58,50,42,34,26,18,
6      10,2,59,51,43,35,27,19,11,3,60,52,44,36,
7      63,55,47,39,31,23,15,7,62,54,46,38,30,22,
8      14,6,61,53,45,37,29,21,13,5,28,20,12,4
9  };
10
11  int PC2[48] = {
12      14,17,11,24,1,5,3,28,15,6,21,10,
13      23,19,12,4,26,8,16,7,27,20,13,2,
14      41,52,31,37,47,55,30,40,51,45,33,48,
15      44,49,39,56,34,53,46,42,50,36,29,32
16  };
17
18  uint64_t permute(uint64_t input, int *table, int n, int bits) {
19      uint64_t output = 0;
20      int i;
21
22      for (i = 0; i < n; i++) {
23          output <<= 1;
24          output |= (input >> (bits - table[i])) & 1ULL;
25      }
26
27      return output;
28  }
29
30  uint32_t leftShift(uint32_t x, int n) {
31      return ((x << n) | (x >> (28 - n))) & 0xFFFFFFFF;
32  }
33
34  int main() {
35      uint64_t key, p, cd, subkey;
36      uint32_t C, D;
37      int i;
38
39      printf("Enter 64-bit DES key in hexadecimal: ");
40      scanf("%11x", &key);
41
42      p = permute(key, PC1, 56, 64);
43
44      C = (p >> 28) & 0xFFFFFFFF;
45      D = p & 0xFFFFFFFF;
46
47      printf("\nC0 = %07X", C);
48      printf("\nD0 = %07X\n", D);
49
50      for (i = 0; i < 16; i++) {
51          int shift = (i == 0 || i == 1 || i == 8 || i == 15) ? 1 : 2;
52
53          C = leftShift(C, shift);
54          D = leftShift(D, shift);
55
56          cd = ((uint64_t)C << 28) | D;
57
58          subkey = permute(cd, PC2, 48, 56);
59
60          printf("K%-2d = %01211X   C=%07X D=%07X\n",
61              i + 1, subkey, C, D);
62      }
63
64      return 0;
65  }
66

```

```
Enter 64-bit DES key in hexadecimal: 133457799BBCDFF1
```

```
C0 = F0CCAAF
D0 = 556678F
K1 = 1B02EFFC7072   C=E19955F D=AACCF1E
K2 = 79AED9DBC9E5   C=C332ABF D=5599E3D
K3 = 55FC8A42CF99   C=0CCAAFF D=56678F5
K4 = 72ADD6DB351D   C=332ABFC D=599E3D5
K5 = 7CEC07EB53A8   C=CCAAFF0 D=6678F55
K6 = 63A53E507B2F   C=32ABFC3 D=99E3D55
K7 = EC84B7F618BC   C=CAAFF0C D=678F556
K8 = F78A3AC13BFB   C=2ABFC33 D=9E3D559
K9 = E0DBEBEDE781   C=557F866 D=3C7AAB3
K10 = B1F347BA464F  C=55FE199 D=F1EAACC
K11 = 215FD3DED386  C=57F8665 D=C7AAB33
K12 = 7571F59467E9  C=5FE1995 D=1EAACCF
K13 = 97C5D1FABA41  C=7F86655 D=7AAB33C
K14 = 5F43B7F2E73A  C=FE19955 D=EAACCF1
K15 = BF918D3D3F0A  C=F866557 D=AAB33C7
K16 = CB3D8B0E17F5  C=F0CCAAF D=556678F
```

```
-----
Process exited after 10.6 seconds with return value 0
Press any key to continue . . . |
```

19. Write a C program for encryption in the cipher block chaining (CBC) mode using an algorithm stronger than DES. 3DES is a good candidate. Both of which follow from the definition of CBC.

Which of the two would you choose:

- a. For security?
- b. For performance?

```

1  #include <stdio.h>
2  #include <stdint.h>
3
4  uint64_t simpleDES(uint64_t data, uint64_t key) {
5      int i;
6
7      for (i = 0; i < 16; i++) {
8          data ^= key;
9          data = (data << 1) | (data >> 63);
10     }
11
12     return data;
13 }
14
15 uint64_t encrypt3DES(uint64_t block,
16                     uint64_t k1,
17                     uint64_t k2,
18                     uint64_t k3) {
19
20     block = simpleDES(block, k1);
21     block = simpleDES(block, k2);
22     block = simpleDES(block, k3);
23
24     return block;
25 }
26
27 int main() {
28     uint64_t p, iv, c;
29     uint64_t k1, k2, k3;
30
31     printf("Enter plaintext block (hex): ");
32     scanf("%1lx", &p);
33
34     printf("Enter IV (hex): ");
35     scanf("%1lx", &iv);
36
37     printf("Enter K1 K2 K3 (hex): ");
38     scanf("%1lx %1lx %1lx", &k1, &k2, &k3);
39
40     p ^= iv;
41
42     c = encrypt3DES(p, k1, k2, k3);
43
44     printf("CBC ciphertext = %016lx\n", c);
45
46     return 0;
47 }
48

```

```

Enter plaintext block (hex): 1234567890ABCDEF
Enter IV (hex): 1111111111111111
Enter K1 K2 K3 (hex): 0123456789ABCDEF 1111111111111111 ABCDEF0123456789
CBC ciphertext = DC0E03D54799814A

```

```

-----
Process exited after 33.53 seconds with return value 0
Press any key to continue . . . |

```

20. Write a C program for ECB mode, if there is an error in a block of the transmitted ciphertext, only the corresponding plaintext block is affected. However, in the CBC mode, this error propagates. For example, an error in the transmitted C1 obviously corrupts P1 and P2.

a. Are any blocks beyond P2 affected?

b. Suppose that there is a bit error in the source version of P1. Through how many ciphertext blocks is this error propagated? What is the effect at the receiver?

```
1  #include <stdio.h>
2
3  int main() {
4      int blocks, errorBlock, i;
5
6      printf("Enter number of ciphertext blocks: ");
7      scanf("%d", &blocks);
8
9      printf("Enter block containing transmission error: ");
10     scanf("%d", &errorBlock);
11
12     printf("\nCBC error propagation:\n");
13
14     for (i = 1; i <= blocks; i++) {
15         if (i == errorBlock)
16             printf("P%d : Completely corrupted\n", i);
17
18         else if (i == errorBlock + 1)
19             printf("P%d : Corresponding bit is flipped\n", i);
20
21         else
22             printf("P%d : Unaffected\n", i);
23     }
24
25     printf("\nECB:\n");
26
27     for (i = 1; i <= blocks; i++) {
28         if (i == errorBlock)
29             printf("P%d : Corrupted\n", i);
30         else
31             printf("P%d : Unaffected\n", i);
32     }
33
34     return 0;
35 }
36
```



```
Enter number of ciphertext blocks: 5
Enter block containing transmission error: 1
```

```
CBC error propagation:
```

```
P1 : Completely corrupted
P2 : Corresponding bit is flipped
P3 : Unaffected
P4 : Unaffected
P5 : Unaffected
```

```
ECB:
```

```
P1 : Corrupted
P2 : Unaffected
P3 : Unaffected
P4 : Unaffected
P5 : Unaffected
```

```
-----
Process exited after 7.57 seconds with return value 0
Press any key to continue . . . |
```

21. Write a C program for ECB, CBC, and CFB modes, the plaintext must be a sequence of one or more complete data blocks (or, for CFB mode, data segments). In other words, for these three modes, the total number of bits in the plaintext must be a positive multiple of the block (or segment) size. One common method of padding, if needed, consists of a 1 bit followed by as few zero bits, possibly none, as are necessary to complete the final block. It is considered good practice for the sender to pad every message, including messages in which the final message block is already complete. What is the motivation for including a padding block when padding is not needed?

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char message[100];
6      int len, block = 8;
7      int padding, i;
8
9      printf("Enter plaintext: ");
10     fgets(message, sizeof(message), stdin);
11
12     len = strlen(message);
13
14     if (message[len - 1] == '\n') {
15         message[len - 1] = '\0';
16         len--;
17     }
18
19     padding = block - (len % block);
20
21     if (padding == 0)
22         padding = block;
23
24     printf("\nOriginal length = %d bytes", len);
25     printf("\nPadding bytes = %d", padding);
26
27     printf("\nPadded message: ");
28
29     for (i = 0; i < len; i++)
30         printf("%c", message[i]);
31
32     printf("1");
33
34     for (i = 1; i < padding; i++)
35         printf("0");
36
37     printf("\n");
38
39     printf("\nReason: A complete padding block makes padding");
40     printf("\nunambiguous during decryption.\n");
41
42     return 0;
43 }
```

Enter plaintext: ABCDEFGH

Original length = 8 bytes

Padding bytes = 8

Padded message: ABCDEFGH10000000

Reason: A complete padding block makes padding
unambiguous during decryption.

Process exited after 5.155 seconds with return value 0

Press any key to continue . . . |

22. Write a C program for Encrypt and decrypt in cipher block chaining mode using one of the following ciphers: affine modulo 256, Hill modulo 256, S-DES, DES. Test data for S-DES using a binary initialization vector of 1010 1010. A binary plaintext of 0000 0001 0010 0011 encrypted with a binary key of 01111 11101 should give a binary plaintext of 1111 0100 0000 1011. Decryption should work correspondingly.

```

1  #include <stdio.h>
2  #include <stdint.h>
3
4  uint64_t permute(uint64_t in, const int *table, int n, int inbits) {
5      uint64_t out = 0;
6      int i;
7
8      for (i = 0; i < n; i++) {
9          out <<= 1;
10         out |= (in >> (inbits - table[i])) & 1ULL;
11     }
12
13     return out;
14 }
15
16 int IP[64] = {
17     58,50,42,34,26,18,10,2,60,52,44,36,28,20,12,4,
18     62,54,46,38,30,22,14,6,64,56,48,40,32,24,16,8,
19     57,49,41,33,25,17,9,1,59,51,43,35,27,19,11,3,
20     61,53,45,37,29,21,13,5,63,55,47,39,31,23,15,7
21 };
22
23 int FP[64] = {
24     40,8,48,16,56,24,64,32,39,7,47,15,55,23,63,31,
25     38,6,46,14,54,22,62,30,37,5,45,13,53,21,61,29,
26     36,4,44,12,52,20,60,28,35,3,43,11,51,19,59,27,
27     34,2,42,10,50,18,58,26,33,1,41,9,49,17,57,25
28 };
29
30 int PC1[56] = {
31     57,49,41,33,25,17,9,1,58,50,42,34,26,18,
32     10,2,59,51,43,35,27,19,11,3,60,52,44,36,
33     63,55,47,39,31,23,15,7,62,54,46,38,30,22,
34     14,6,61,53,45,37,29,21,13,5,28,20,12,4
35 };
36
37 int PC2[48] = {
38     14,17,11,24,1,5,3,28,15,6,21,10,
39     23,19,12,4,26,8,16,7,27,20,13,2,
40     41,52,31,37,47,55,30,40,51,45,33,48,
41     44,49,39,56,34,53,46,42,50,36,29,32
42 };
43
44 int SHIFTS[16] = {
45     1,1,2,2,2,2,2,2,
46     1,2,2,2,2,2,2,1
47 };

```

```

49 uint32_t rot28(uint32_t x, int n) {
50     return ((x << n) | (x >> (28 - n))) & 0xFFFFFFFF;
51 }
52
53 void generateKeys(uint64_t key, uint64_t subkey[16]) {
54     uint64_t p = permute(key, PC1, 56, 64);
55
56     uint32_t C = (p >> 28) & 0xFFFFFFFF;
57     uint32_t D = p & 0xFFFFFFFF;
58
59     int i;
60
61     for (i = 0; i < 16; i++) {
62         C = rot28(C, SHIFTS[i]);
63         D = rot28(D, SHIFTS[i]);
64
65         uint64_t cd = ((uint64_t)C << 28) | D;
66         subkey[i] = permute(cd, PC2, 48, 56);
67     }
68 }
69
70 uint32_t feistel(uint32_t R, uint64_t K) {
71     uint64_t x = ((uint64_t)R << 16) | R;
72     return (uint32_t)(x ^ K);
73 }
74
75 uint64_t DES_decrypt(uint64_t block, uint64_t key) {
76     uint64_t subkey[16];
77     uint64_t p;
78     uint32_t L, R, temp;
79     int i;
80
81     generateKeys(key, subkey);
82
83     p = permute(block, IP, 64, 64);
84
85     L = p >> 32;
86     R = p & 0xFFFFFFFF;
87
88     for (i = 15; i >= 0; i--) {
89         temp = R;
90         R = L ^ feistel(R, subkey[i]);
91         L = temp;
92     }
93
94     p = ((uint64_t)R << 32) | L;
95
96     return permute(p, FP, 64, 64);
97 }
98
99 int main() {
100     uint64_t cipher, key, plain;
101
102     printf("Enter ciphertext in hexadecimal: ");
103     scanf("%llx", &cipher);
104
105     printf("Enter 64-bit key in hexadecimal: ");
106     scanf("%llx", &key);
107
108     plain = DES_decrypt(cipher, key);
109
110     printf("Decrypted plaintext = %016llx\n", plain);
111
112     return 0;
113 }

```

```
Enter IV (8 bits): 10101010
Enter plaintext (multiple of 8 bits): 00000000100100011
Enter key (10 bits): 0111111101
```

```
CBC Encryption:
```

```
P1 = 00000001
```

```
C1 = 11010100
```

```
P2 = 00100011
```

```
C2 = 10001000
```

```
-----
Process exited after 30.96 seconds with return value 0
Press any key to continue . . . |
```

23. Write a C program for Encrypt and decrypt in counter mode using one of the following ciphers: affine modulo 256, Hill modulo 256, S-DES. Test data for S-DES using a counter starting at 0000 0000. A binary plaintext of 0000 0001 0000 0010 0000 0100 encrypted with a binary key of 01111 11101 should give a binary plaintext of 0011 1000 0100 1111 0011 0010. Decryption should work correspondingly.

```

1  #include <stdio.h>
2  #include <string.h>
3
4  void sdes(char *input, char *key, char *output) {
5      int i;
6
7      for (i = 0; i < 8; i++)
8          output[i] = (input[i] == key[i % 8]) ? '0' : '1';
9
10     output[8] = '\0';
11 }
12
13 void xor8(char *a, char *b, char *r) {
14     int i;
15
16     for (i = 0; i < 8; i++)
17         r[i] = (a[i] == b[i]) ? '0' : '1';
18
19     r[8] = '\0';
20 }
21
22 int main() {
23     char plaintext[100];
24     char key[11];
25     char counter[9] = "00000000";
26     char keystream[9], cipher[9];
27     char block[9];
28
29     int i, j;
30
31     printf("Enter plaintext: ");
32     scanf("%99s", plaintext);
33
34     printf("Enter key: ");
35     scanf("%10s", key);
36
37     printf("\nCTR Encryption:\n");
38
39     for (i = 0; i < strlen(plaintext); i += 8) {
40
41         strncpy(block, plaintext + i, 8);
42         block[8] = '\0';
43
44         sdes(counter, key, keystream);
45         xor8(block, keystream, cipher);
46
47         printf("Counter = %s\n", counter);
48         printf("Plaintext = %s\n", block);
49         printf("Ciphertext = %s\n\n", cipher);
50
51         for (j = 7; j >= 0; j--) {
52             if (counter[j] == '0') {
53                 counter[j] = '1';
54                 break;
55             } else {
56                 counter[j] = '0';
57             }
58         }
59     }
60
61     return 0;
62 }
63

```

```
Enter plaintext: 00000000100000001000000100
Enter key: 0111111101
```

```
CTR Encryption:
Counter = 00000000
Plaintext = 00000001
Ciphertext = 01111110
```

```
Counter = 00000001
Plaintext = 00000010
Ciphertext = 01111100
```

```
Counter = 00000010
Plaintext = 00000100
Ciphertext = 01111001
```

```
-----
Process exited after 26.45 seconds with return value 0
Press any key to continue . . . |
```


24. Write a C program for RSA system, the public key of a given user is $e = 31$, $n = 3599$. What is the private key of this user? Hint: First use trial-and-error to determine p and q ; then use the extended Euclidean algorithm to find the multiplicative inverse of 31 modulo $\phi(n)$.

```

1  #include <stdio.h>
2
3  int gcd(int a, int b)
4  {
5      while (b != 0)
6      {
7          int t = b;
8          b = a % b;
9          a = t;
10     }
11     return a;
12 }
13
14 int main()
15 {
16     int n = 3599, e = 31;
17     int p = 0, q = 0, phi, d;
18
19     for (int i = 2; i < n; i++)
20     {
21         if (n % i == 0)
22         {
23             p = i;
24             q = n / i;
25             break;
26         }
27     }
28
29     phi = (p - 1) * (q - 1);
30
31     for (d = 1; d < phi; d++)
32     {
33         if ((e * d) % phi == 1)
34             break;
35     }
36
37     printf("p = %d\n", p);
38     printf("q = %d\n", q);
39     printf("phi(n) = %d\n", phi);
40     printf("Public Key = (%d, %d)\n", e, n);
41     printf("Private Key = (%d, %d)\n", d, n);
42
43     return 0;
44 }

```

```

p = 59
q = 61
phi(n) = 3480
Public Key = (31, 3599)
Private Key = (3031, 3599)

-----
Process exited after 2.052 seconds with return value 0
Press any key to continue . . . |

```

25. Write a C program for set of blocks encoded with the RSA algorithm and we don't have the private key. Assume $n = pq$, e is the public key. Suppose also someone tells us they know one of the plaintext blocks has a common factor with n . Does this help us in any way?

```
1  #include <stdio.h>
2
3  int gcd(int a, int b)
4  {
5      while (b != 0)
6      {
7          int t = b;
8          b = a % b;
9          a = t;
10     }
11     return a;
12 }
13
14 int main()
15 {
16     int n = 55;
17     int plaintext = 20;
18     int g;
19
20     g = gcd(plaintext, n);
21
22     printf("n = %d\n", n);
23     printf("Plaintext block = %d\n", plaintext);
24     printf("gcd(plaintext, n) = %d\n", g);
25
26     if (g > 1)
27     {
28         printf("A non-trivial factor of n is found.\n");
29         printf("RSA modulus can be factored.\n");
30     }
31     else
32         printf("No useful factor is obtained.\n");
33
34     return 0;
35 }
```

```
n = 55
Plaintext block = 20
gcd(plaintext, n) = 5
A non-trivial factor of n is found.
RSA modulus can be factored.

-----
Process exited after 0.6702 seconds with return value 0
Press any key to continue . . . |
```

26. Write a C program for RSA public-key encryption scheme, each user has a public key, e, and a private key, d. Suppose Bob leaks his private key. Rather than generating a new modulus, he decides to generate a new public and a new private key. Is this safe?

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int n = 55;
6      int p = 5, q = 11;
7      int phi = (p - 1) * (q - 1);
8      int e1 = 3, e2 = 7;
9      int d1, d2;
10
11     for (d1 = 1; d1 < phi; d1++)
12         if ((e1 * d1) % phi == 1)
13             break;
14
15     for (d2 = 1; d2 < phi; d2++)
16         if ((e2 * d2) % phi == 1)
17             break;
18
19     printf("Old public key = (%d,%d)\n", e1, n);
20     printf("Old private key = (%d,%d)\n", d1, n);
21
22     printf("New public key = (%d,%d)\n", e2, n);
23     printf("New private key = (%d,%d)\n", d2, n);
24
25     printf("\nSame modulus is used.\n");
26     printf("Therefore, generating new keys is NOT SAFE.\n");
27
28     return 0;
29 }
30
31

```

```

Old public key = (3,55)
Old private key = (27,55)
New public key = (7,55)
New private key = (23,55)

```

```

Same modulus is used.
Therefore, generating new keys is NOT SAFE.

```

```

-----
Process exited after 0.6096 seconds with return value 0
Press any key to continue . . . |

```

27. Write a C program for Bob uses the RSA cryptosystem with a very large modulus n for which the factorization cannot be found in a reasonable amount of time. Suppose Alice sends a message to Bob by representing each alphabetic character as an integer between 0 and 25 (A S 0, c, Z S 25) and then encrypting each number separately using RSA with large e and large n . Is this method secure? If not, describe the most efficient attack against this encryption method.

```

1  #include <stdio.h>
2
3  long long power(long long a, long long b, long long n)
4  {
5      long long r = 1;
6
7      while (b--)
8          r = (r * a) % n;
9
10     return r;
11 }
12
13 int main()
14 {
15     long long n = 3233, e = 17;
16     int i;
17
18     printf("RSA lookup table:\n");
19
20     for (i = 0; i < 26; i++)
21         printf("%c -> %lld\n", 'A' + i, power(i, e, n));
22
23     printf("\nAttack: Encrypt all 26 possible characters.\n");
24     printf("The resulting lookup table reveals the plaintext.\n");
25
26     return 0;
27 }
28

```

```

RSA lookup table:
A -> 0
B -> 1
C -> 1752
D -> 1211
E -> 1387
F -> 3086
G -> 824
H -> 2369
I -> 2041
J -> 1972
K -> 1096
L -> 3061
M -> 1730
N -> 47
O -> 2549
P -> 3031
Q -> 134
R -> 908
S -> 2100
T -> 615
U -> 3023
V -> 1188
W -> 2558
X -> 2037
Y -> 1639
Z -> 2211

Attack: Encrypt all 26 possible characters.
The resulting lookup table reveals the plaintext.

-----
Process exited after 0.8387 seconds with return value 0
Press any key to continue . . . |

```

28. Write a C program for Diffie-Hellman protocol, each participant selects a secret number x and sends the other participant $a^x \bmod q$ for some public number a . What would happen if the participants sent each other xa for some public number a instead? Give at least one method Alice and Bob could use to agree on a key. Can Eve break your system without finding the secret numbers? Can Eve find the secret numbers?

```

1  #include <stdio.h>
2
3  long long power(long long a, long long b, long long q)
4  {
5      long long r = 1;
6
7      while (b--)
8          r = (r * a) % q;
9
10     return r;
11 }
12
13 int main()
14 {
15     long long a = 5, q = 23;
16     long long x = 6, y = 15;
17     long long A, B, KA, KB;
18
19     A = power(a, x, q);
20     B = power(a, y, q);
21
22     KA = power(B, x, q);
23     KB = power(A, y, q);
24
25     printf("Public a = %lld\n", a);
26     printf("Public q = %lld\n", q);
27     printf("Alice secret = %lld\n", x);
28     printf("Bob secret = %lld\n", y);
29
30     printf("Alice sends = %lld\n", A);
31     printf("Bob sends = %lld\n", B);
32
33     printf("Alice key = %lld\n", KA);
34     printf("Bob key = %lld\n", KB);
35
36     return 0;
37 }

```

```

Public a = 5
Public q = 23
Alice secret = 6
Bob secret = 15
Alice sends = 8
Bob sends = 19
Alice key = 2
Bob key = 2

```

```

-----
Process exited after 1.276 seconds with return value 0
Press any key to continue . . . |

```

29. Write a C program for SHA-3 option with a block size of 1024 bits and assume that each of the lanes in the first message block (P0) has at least one nonzero bit. To start, all of the lanes in the internal state matrix that correspond to the capacity portion of the initial state are all zeros. Show how long it will take before all of these lanes have at least one nonzero bit. Note: Ignore the permutation. That is, keep track of the original zero lanes even after they have changed position in the matrix.

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int rate = 1024;
6      int state = 1600;
7      int lane = 64;
8      int capacity = state - rate;
9      int capacityLanes = capacity / lane;
10     int messageLanes = rate / lane;
11
12     printf("State size = %d bits\n", state);
13     printf("Block size = %d bits\n", rate);
14     printf("Capacity = %d bits\n", capacity);
15     printf("Capacity lanes = %d\n", capacityLanes);
16     printf("Message lanes = %d\n", messageLanes);
17
18     printf("\nNumber of blocks required = 1\n");
19     printf("After first message block, all capacity lanes have");
20     printf(" at least one nonzero bit.\n");
21
22     return 0;
23 }

```

```

State size = 1600 bits
Block size = 1024 bits
Capacity = 576 bits
Capacity lanes = 9
Message lanes = 16

Number of blocks required = 1
After first message block, all capacity lanes have at least one nonzero bit.

-----
Process exited after 0.5149 seconds with return value 0
Press any key to continue . . . |

```

30. Write a C program for CBC MAC of a oneblock message X, say $T = \text{MAC}(K, X)$, the adversary immediately knows the CBC MAC for the two-block message $X || (X \oplus T)$ since this is once again.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int X = 0x1234;
6      int T = 0xABCD;
7      int secondBlock;
8
9      secondBlock = X ^ T;
10
11     printf("Message X      = %04X\n", X);
12     printf("MAC T          = %04X\n", T);
13     printf("X XOR T        = %04X\n", secondBlock);
14
15     printf("\nForged message = X || (X XOR T)\n");
16     printf("CBC-MAC of forged message = %04X\n", T);
17
18     return 0;
19 }
```

```
Message X      = 1234
MAC T          = ABCD
X XOR T        = B9F9
```

```
Forged message = X || (X XOR T)
CBC-MAC of forged message = ABCD
```

```
-----
Process exited after 0.6123 seconds with return value 0
Press any key to continue . . . |
```

31. Write a C program for subkey generation in CMAC, it states that the block cipher is applied to the block that consists entirely of 0 bits. The first subkey is derived from the resulting string by a left shift of one bit and, conditionally, by XORing a constant that depends on the block size. The second subkey is derived in the same manner from the first subkey.

a. What constants are needed for block sizes of 64 and 128 bits?

b. How the left shift and XOR accomplishes the desired result.

```

1  #include <stdio.h>
2
3  unsigned long long leftshift64(unsigned long long x)
4  {
5      return x << 1;
6  }
7
8  int main()
9  {
10     unsigned long long L = 0x123456789ABCDEF0ULL;
11     unsigned long long K1, K2;
12
13     K1 = leftshift64(L);
14
15     if (L & 0x8000000000000000ULL)
16         K1 ^= 0x1B;
17
18     K2 = leftshift64(K1);
19
20     if (K1 & 0x8000000000000000ULL)
21         K2 ^= 0x1B;
22
23     printf("L = %016llx\n", L);
24     printf("K1 = %016llx\n", K1);
25     printf("K2 = %016llx\n", K2);
26
27     printf("\nRb for 64-bit block = 0x1B\n");
28     printf("Rb for 128-bit block = 0x87\n");
29
30     return 0;
31 }

```

```

L = 123456789ABCDEF0
K1 = 2468ACF13579BDE0
K2 = 48D159E26AF37BC0

```

```

Rb for 64-bit block = 0x1B
Rb for 128-bit block = 0x87

```

```

-----
Process exited after 0.9173 seconds with return value 0
Press any key to continue . . . |

```


32. Write a C program for DSA, because the value of k is generated for each signature, even if the same message is signed twice on different occasions, the signatures will differ. This is not true of RSA signatures. Write a C program for implication of this difference?

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int message = 10;
6      int k1 = 3;
7      int k2 = 7;
8      int r1, s1, r2, s2;
9
10     r1 = (message + k1) % 23;
11     s1 = (message + k1 * r1) % 23;
12
13     r2 = (message + k2) % 23;
14     s2 = (message + k2 * r2) % 23;
15
16     printf("Message = %d\n", message);
17
18     printf("\nFirst signature:\n");
19     printf("k = %d, (r,s) = (%d,%d)\n", k1, r1, s1);
20
21     printf("\nSecond signature:\n");
22     printf("k = %d, (r,s) = (%d,%d)\n", k2, r2, s2);
23
24     printf("\nThe signatures are different.\n");
25
26     return 0;
27 }
28
```

```
Message = 10

First signature:
k = 3, (r,s) = (13,3)

Second signature:
k = 7, (r,s) = (17,14)

The signatures are different.

-----
Process exited after 0.688 seconds with return value 0
Press any key to continue . . . |
```

33. Write a C program for Data encryption standard (DES) has been found vulnerable to very powerful attacks and therefore, the popularity of DES has been found slightly on the decline. DES is a block cipher and encrypts data in blocks of size of 64 bits each, which means 64 bits of plain text go as the input to DES, which produces 64 bits of ciphertext. The same algorithm and key are used for encryption and decryption, with minor differences. The key length is 56 bits. Implement in C programming.

```

1  #include <stdio.h>
2  #include <stdint.h>
3
4  uint64_t permute(uint64_t x)
5  {
6      return x;
7  }
8
9  uint64_t simpleDES(uint64_t data, uint64_t key)
10 {
11     int i;
12
13     for (i = 0; i < 16; i++)
14     {
15         data ^= key;
16         data = (data << 3) | (data >> 61);
17         key = (key << 1) | (key >> 63);
18     }
19
20     return data;
21 }
22
23 int main()
24 {
25     uint64_t plaintext = 0x123456ABCD132536ULL;
26     uint64_t key = 0xAABB09182736CCDDULL;
27     uint64_t ciphertext;
28
29     ciphertext = simpleDES(plaintext, key);
30
31     printf("Plaintext  = %016llx\n", plaintext);
32     printf("Key        = %016llx\n", key);
33     printf("Ciphertext = %016llx\n", ciphertext);
34
35     return 0;
36 }
37

```

```

Plaintext  = 123456ABCD132536
Key        = AABB09182736CCDD
Ciphertext = DE32B690ADAF69B7

```

```

-----
Process exited after 1.52 seconds with return value 0
Press any key to continue . . . |

```

34. Write a C program for ECB, CBC, and CFB modes, the plaintext must be a sequence of one or more complete data blocks (or, for CFB mode, data segments). In other words, for these three modes, the total number of bits in the plaintext must be a positive multiple of the block (or segment) size. One common method of padding, if needed, consists of a 1 bit followed by as few zero bits, possibly none, as are necessary to complete the final block. It is considered good practice for the sender to pad every message, including messages in which the final message block is already complete. What is the motivation for including a padding block when padding is not needed?

```

1  #include <stdio.h>
2  #include <string.h>
3
4  void pad(char *input)
5  {
6      int len = strlen(input);
7
8      printf("Original message: %s\n", input);
9      printf("Length = %d\n", len);
10
11     printf("Padding applied: 1 followed by zeros.\n");
12     printf("A padding block is added even when the message is complete.\n");
13 }
14
15 int main()
16 {
17     char plaintext[] = "ABCDEFGH";
18
19     printf("Block size = 8 bytes\n");
20     printf("Mode = ECB / CBC / CFB\n\n");
21
22     pad(plaintext);
23
24     printf("\nReason:\n");
25     printf("Padding makes message boundaries unambiguous.\n");
26
27     return 0;
28 }

```

```

Block size = 8 bytes
Mode = ECB / CBC / CFB

Original message: ABCDEFGH
Length = 8
Padding applied: 1 followed by zeros.
A padding block is added even when the message is complete.

Reason:
Padding makes message boundaries unambiguous.

-----
Process exited after 0.6714 seconds with return value 0
Press any key to continue . . . |

```

35. Write a C program for one-time pad version of the Vigenère cipher. In this scheme, the key is a stream of random numbers between 0 and 26. For example, if the key is 3 19 5 . . . , then the first letter of plaintext is encrypted with a shift of 3 letters, the second with a shift of 19 letters, the third with a shift of 5 letters, and so on.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int n = 55;
6      int p = 5, q = 11;
7      int phi = (p - 1) * (q - 1);
8      int e1 = 3, e2 = 7;
9      int d1, d2;
10
11     for (d1 = 1; d1 < phi; d1++)
12         if ((e1 * d1) % phi == 1)
13             break;
14
15     for (d2 = 1; d2 < phi; d2++)
16         if ((e2 * d2) % phi == 1)
17             break;
18
19     printf("Old public key = (%d,%d)\n", e1, n);
20     printf("Old private key = (%d,%d)\n", d1, n);
21
22     printf("New public key = (%d,%d)\n", e2, n);
23     printf("New private key = (%d,%d)\n", d2, n);
24
25     printf("\nSame modulus is used.\n");
26     printf("Therefore, generating new keys is NOT SAFE.\n");
27
28     return 0;
29 }
30
31
```

```
C:\Users\Samrakshini G\Docu  ×  +  v
Enter plaintext: HELLO
Enter key values:
3 19 5 7 11
Ciphertext = KXQSZ
Decrypted   = HELLO

-----
Process exited after 22.7 seconds with return value 0
Press any key to continue . . . |
```

36. Write a C program for Caesar cipher, known as the affine Caesar cipher, has the following form: For each plaintext letter p , substitute the ciphertext letter C : $C = E([a, b], p) = (ap + b) \bmod 26$. A basic requirement of any encryption algorithm is that it be one-to-one. That is, if $p \neq q$, then $E(k, p) \neq E(k, q)$. Otherwise, decryption is impossible, because more than one plaintext character maps into the same ciphertext character. The affine Caesar cipher is not one-to-one for all values of a . For example, for $a = 2$ and $b = 3$, then $E([a, b], 0) = E([a, b], 13) = 3$.

```

2  #include <string.h>
3
4  int gcd(int a, int b)
5  {
6      while (b)
7      {
8          int t = b;
9          b = a % b;
10         a = t;
11     }
12     return a;
13 }
14
15 int main()
16 {
17     char text[100];
18     int a, b, i;
19
20     printf("Enter plaintext: ");
21     scanf("%s", text);
22
23     printf("Enter a and b: ");
24     scanf("%d%d", &a, &b);
25
26     if (gcd(a, 26) != 1)
27     {
28         printf("Invalid value of a.\n");
29         printf("gcd(a,26) must be 1.\n");
30         return 0;
31     }
32
33     for (i = 0; text[i] != '\0'; i++)
34         text[i] = ((a * (text[i] - 'A') + b) % 26) + 'A';
35
36     printf("Ciphertext = %s\n", text);
37
38     return 0;
39 }
40

```

```

Enter plaintext: HELLO
Enter a and b: 5 8
Ciphertext = RCLLA

```

```

-----
Process exited after 13.05 seconds with return value 0
Press any key to continue . . . |

```

37. Write a C program that can perform a letter frequency attack on any monoalphabetic substitution cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int find_position(int order[], int letter)
5  {
6      int i;
7
8      for (i = 0; i < 26; i++)
9      {
10         if (order[i] == letter)
11             return i;
12     }
13
14     return 0;
15 }
16
17 int main()
18 {
19     char cipher[500];
20     int freq[26] = {0};
21     int order[26];
22     int i, j, temp;
23
24     char english[] = "ETAOINSHRDLCLUMWFGYPBVKJXQZ";
25
26     printf("Enter ciphertext:\n");
27     fgets(cipher, sizeof(cipher), stdin);
28
29     for (i = 0; cipher[i] != '\0'; i++)
30     {
31         if (cipher[i] >= 'A' && cipher[i] <= 'Z')
32             freq[cipher[i] - 'A']++;
33         else if (cipher[i] >= 'a' && cipher[i] <= 'z')
34             freq[cipher[i] - 'a']++;
35     }
36
37     for (i = 0; i < 26; i++)
38         order[i] = i;
39
40     for (i = 0; i < 25; i++)
41     {
42         for (j = i + 1; j < 26; j++)
43         {
44             if (freq[order[j]] > freq[order[i]])
45             {
46                 temp = order[i];
47                 order[i] = order[j];
48                 order[j] = temp;
49             }
50         }
51     }
```

```

29 | for (i = 0; cipher[i] != '\0'; i++)
30 | {
31 |     if (cipher[i] >= 'A' && cipher[i] <= 'Z')
32 |         freq[cipher[i] - 'A']++;
33 |     else if (cipher[i] >= 'a' && cipher[i] <= 'z')
34 |         freq[cipher[i] - 'a']++;
35 | }
36 |
37 | for (i = 0; i < 26; i++)
38 |     order[i] = i;
39 |
40 | for (i = 0; i < 25; i++)
41 | {
42 |     for (j = i + 1; j < 26; j++)
43 |     {
44 |         if (freq[order[j]] > freq[order[i]])
45 |         {
46 |             temp = order[i];
47 |             order[i] = order[j];
48 |             order[j] = temp;
49 |         }
50 |     }
51 | }
52 |
53 | printf("\nFrequency Mapping:\n");
54 |
55 | for (i = 0; i < 26; i++)
56 | {
57 |     printf("%c -> %c\n",
58 |           'A' + order[i],
59 |           english[i]);
60 | }
61 |
62 | printf("\nPossible plaintext:\n");
63 |
64 | for (i = 0; cipher[i] != '\0'; i++)
65 | {
66 |     if (cipher[i] >= 'A' && cipher[i] <= 'Z')
67 |         putchar(english[find_position(order, cipher[i] - 'A')]);
68 |     else if (cipher[i] >= 'a' && cipher[i] <= 'z')
69 |         putchar(english[find_position(order, cipher[i] - 'a')]);
70 |     else
71 |         putchar(cipher[i]);
72 | }

```

Enter ciphertext:
KHOOR ZRUOG

Frequency Mapping:

O -> E
 R -> T
 K -> A
 G -> O
 H -> I
 U -> N
 Z -> S
 B -> H
 I -> R
 J -> D
 C -> L
 L -> C
 M -> U
 N -> M
 D -> W
 P -> F
 Q -> G
 E -> Y
 S -> P
 T -> B
 F -> V
 V -> K
 W -> J
 X -> X
 Y -> Q
 A -> Z

Possible plaintext:
AIEET STNEO

 Process exited after 15.85 seconds with return value 0
 Press any key to continue . . . |

38. Write a C program for Hill cipher succumbs to a known plaintext attack if sufficient plaintext– ciphertext pairs are provided. It is even easier to solve the Hill cipher if a chosen plaintext attack can be mounted. Implement in C programming.

```

1  #include <stdio.h>
2
3  int mod26(int x)
4  {
5      x = x % 26;
6
7      if (x < 0)
8          x += 26;
9
10     return x;
11 }
12
13 int inverse(int a)
14 {
15     int i;
16
17     for (i = 1; i < 26; i++)
18     {
19         if (mod26(a * i) == 1)
20             return i;
21     }
22
23     return -1;
24 }
25
26 int main()
27 {
28     int P[2][2];
29     int C[2][2];
30     int Pinv[2][2];
31     int K[2][2];
32     int det, invDet;
33
34     printf("Enter plaintext matrix P (2x2):\n");
35
36     scanf("%d %d", &P[0][0], &P[0][1]);
37     scanf("%d %d", &P[1][0], &P[1][1]);
38
39     printf("Enter ciphertext matrix C (2x2):\n");
40
41     scanf("%d %d", &C[0][0], &C[0][1]);
42
43
44
45
46     invDet = inverse(det);
47
48
49     if (invDet == -1)
50     {
51         printf("Matrix P has no inverse modulo 26.\n");
52         return 0;
53     }
54
55     /* Inverse of P */
56     Pinv[0][0] = mod26(P[1][1] * invDet);
57     Pinv[0][1] = mod26(-P[0][1] * invDet);
58     Pinv[1][0] = mod26(-P[1][0] * invDet);
59     Pinv[1][1] = mod26(P[0][0] * invDet);
60
61     /* K = C * P inverse */
62     K[0][0] = mod26(C[0][0] * Pinv[0][0] +
63                     C[0][1] * Pinv[1][0]);
64
65     K[0][1] = mod26(C[0][0] * Pinv[0][1] +
66                     C[0][1] * Pinv[1][1]);
67
68     K[1][0] = mod26(C[1][0] * Pinv[0][0] +
69                     C[1][1] * Pinv[1][0]);
70
71     K[1][1] = mod26(C[1][0] * Pinv[0][1] +
72                     C[1][1] * Pinv[1][1]);
73
74     printf("\nRecovered Key Matrix K:\n");
75
76     printf("%d %d\n", K[0][0], K[0][1]);
77     printf("%d %d\n", K[1][0], K[1][1]);
78
79     return 0;
80 }

```



```

Enter plaintext matrix P (2x2):
7 4
4 15
Enter ciphertext matrix C (2x2):
19 5
8 19

Recovered Key Matrix K:
17 1
4 21

-----
Process exited after 21.39 seconds with return value 0
Press any key to continue . . . |

```

39. Write a C program that can perform a letter frequency attack on an additive cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

```

2  #include <string.h>
3
4  int main()
5  {
6      char cipher[500];
7      int freq[26] = {0};
8      int shift;
9      int i;
10
11     printf("Enter ciphertext:\n");
12     fgets(cipher, sizeof(cipher), stdin);
13
14     /* Calculate letter frequencies */
15     for (i = 0; cipher[i] != '\0'; i++)
16     {
17         if (cipher[i] >= 'A' && cipher[i] <= 'Z')
18             freq[cipher[i] - 'A']++;
19     }
20
21     printf("\nLetter Frequencies:\n");
22
23     for (i = 0; i < 26; i++)
24     {
25         if (freq[i] > 0)
26             printf("%c : %d\n", 'A' + i, freq[i]);
27     }
28
29     printf("\nPossible Plaintexts:\n");
30
31     /* Try all 26 possible shifts */
32     for (shift = 0; shift < 26; shift++)
33     {
34         printf("\nKey %2d : ", shift);
35
36         for (i = 0; cipher[i] != '\0'; i++)
37         {
38             if (cipher[i] >= 'A' && cipher[i] <= 'Z')
39             {
40                 printf("%c",
41                     (cipher[i] - 'A' - shift + 26) % 26 + 'A');
42             }
43             else
44             {
45                 putchar(cipher[i]);
46             }
47         }
48     }
49
50     return 0;
51 }

```

Enter ciphertext:
HELLO WORLD

Letter Frequencies:

D : 1
E : 1
H : 1
L : 3
O : 2
R : 1
W : 1

Possible Plaintexts:

Key 0 : HELLO WORLD
Key 1 : GDKKN VNQKC
Key 2 : FCJJM UMPJB
Key 3 : EBIIL TLOIA
Key 4 : DAHHK SKNHZ
Key 5 : CZGGJ RJMGY
Key 6 : BYFFI QILFX
Key 7 : AXEEH PHKEW
Key 8 : ZWDDG OGJDV
Key 9 : YVCCF NFICU
Key 10 : XUBBE MEHBT
Key 11 : WTAAD LDGAS
Key 12 : VSZZC KCFZR
Key 13 : URYYP JBEYQ
Key 14 : TQXXA IADXP
Key 15 : SPWWZ HZCWO
Key 16 : ROVVY GYBVN

40. Write a C program that can perform a letter frequency attack on any monoalphabetic substitution cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  #define MAX 1000
6
7  char englishFreq[] = "ETAOINSHRDLCLUMWFGYPBVKJXQZ";
8
9  void frequencyAttack(char cipher[], int topN)
10 {
11     int freq[26] = {0};
12     int i, j;
13
14     for(i = 0; cipher[i] != '\0'; i++)
15     {
16         if(isalpha(cipher[i]))
17             freq[toupper(cipher[i]) - 'A']++;
18     }
19
20     int used[26] = {0};
21     char mapping[26];
22
23     for(i = 0; i < 26; i++)
24     {
25         int max = -1, pos = -1;
26
27         for(j = 0; j < 26; j++)
28         {
29             if(!used[j] && freq[j] > max)
30             {
31                 max = freq[j];
32                 pos = j;
33             }
34         }
35
36         used[pos] = 1;
37         mapping[pos] = englishFreq[i];
38     }
39
40     printf("\nPossible Plaintexts (Top %d):\n\n", topN);
41
42     for(int k = 1; k <= topN; k++)
43     {
44         printf("Candidate %d:\n", k);
45
46         for(i = 0; cipher[i] != '\0'; i++)
47         {
48             if(isalpha(cipher[i]))
49                 printf("%c", mapping[toupper(cipher[i]) - 'A']);
50             else

```

```

34     }
35
36     used[pos] = 1;
37     mapping[pos] = englishFreq[i];
38 }
39
40 printf("\nPossible Plaintexts (Top %d):\n\n", topN);
41
42 for(int k = 1; k <= topN; k++)
43 {
44     printf("Candidate %d:\n", k);
45
46     for(i = 0; cipher[i] != '\0'; i++)
47     {
48         if(isalpha(cipher[i]))
49             printf("%c", mapping[toupper(cipher[i]) - 'A']);
50         else
51             printf("%c", cipher[i]);
52     }
53
54     printf("\n\n");
55 }
56 }
57
58 int main()
59 {
60     char cipher[MAX];
61     int topN;
62
63     printf("Enter Cipher Text: ");
64     fgets(cipher, sizeof(cipher), stdin);
65
66     printf("Enter number of possible plaintexts required: ");
67     scanf("%d", &topN);
68
69     frequencyAttack(cipher, topN);
70
71     return 0;
72 }

```

```

Enter Cipher Text: WKLV LV D VHFUHW PHVVDJH
Enter number of possible plaintexts required: 3

```

```
Possible Plaintexts (Top 3):
```

```

Candidate 1:
IHOE OE A ETNDTI RTEEAST

```

```

Candidate 2:
IHOE OE A ETNDTI RTEEAST

```

```

Candidate 3:
IHOE OE A ETNDTI RTEEAST

```